Zemen: Dual-Calendar Web Application
Software Engineering - Final Project Report

By Aklesia Berihu Mekonnen and Meron Zemichael Kahsay
Programme: 4th Year Data Analysis

1. Introduction

When we first proposed this project, our idea was straightforward: build a web app that lets people manage events across both the Ethiopian and Gregorian calendar systems. It sounds simple enough, but the further we got into development, the more we realised how much interesting complexity was hiding underneath that idea. Managing dates across two calendar systems with different year lengths, different New Year dates, and a 13th month is already non-trivial. Add collaborative event editing, concurrent modifications, smart scheduling across multiple participants, and timezone handling, and you have a system with real algorithmic depth.

The application is called **Zemen** (the Amharic word for "era" or "time"). It is a full-stack collaborative scheduling platform that serves anyone who needs to work across both calendar systems like Ethiopian students abroad, international teams collaborating with Ethiopian partners, or simply anyone who wants to plan around Ethiopian public holidays. The final system delivers everything we originally proposed, plus the algorithmically richer features the professor asked us to develop shared event collaboration with version conflict detection, an interval-based scheduling engine, and a constraint-based slot ranking algorithm.

This report documents what we built, how we built it, the design decisions we made, and the evidence that it all works correctly.

2. System Architecture

Zemen is built as a three-tier web application deployed using Docker Compose, which means the whole system; frontend, backend, and database spins up with a single

command. The architecture was designed to be clean and modular from the start, which made it easy to add features across sprints without breaking things.

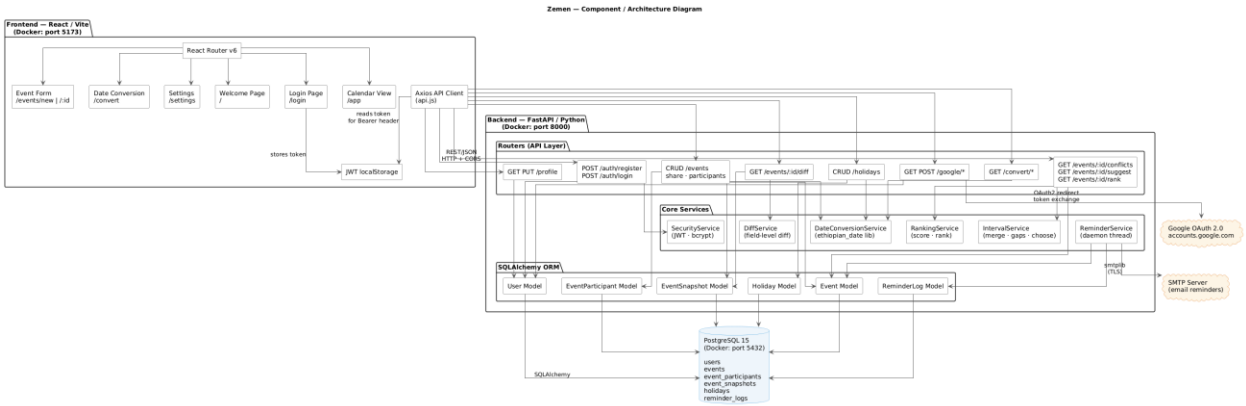

Figure 1: System component diagram showing the frontend, backend, database, and external service integrations.

The layers are:

- **Frontend - React 18 with Vite, running on port 5173:** We used React Router v6 for navigation and Axios for all API calls. The frontend stores the JWT in localStorage and attaches it as a Bearer token on every authenticated request.

- **Backend - FastAPI (Python 3.12), running on port 8000:** The API is organised into focused routers; auth, profile, events, conversion, scheduling, diff, holidays, and Google, each handling its own domain. Core business logic lives in separate service modules so the routers stay thin.

- **Database - PostgreSQL 15, with six tables: ‘**users’, ‘events’, ‘event_participants’, ‘event_snapshots’, ‘holidays’, and ‘reminder_logs’. The schema was designed in Sprint 1 and held up throughout the entire project without needing structural changes.

- **External services:** Google OAuth 2.0 for authentication and calendar export, and an SMTP server for email reminders. Both are optional, the system degrades gracefully if they are not configured.

3. Database Design

The data model reflects the collaborative nature of the application. Events have an owner (‘user_id’ foreign key) but can also have multiple participants through the ‘event_participants’ junction table, which stores each participant's role (owner, editor, or viewer). Every time an event is updated, a snapshot of its state is written to ‘event_snapshots’, this is what makes the version history and field-level diff features possible. The ‘reminder_logs’ table prevents duplicate emails by recording each reminder that has already been sent.

Tabel	Purpose
Users	Accounts, preferences, Google OAuth tokens
events	Core event data, stored in UTC with version counter
event_participants	Role-based sharing (owner / editor / viewer)
event_snapshots	Immutable audit trail -one row per event version
holidays	Ethiopian and Gregorian public holidays
reminder_logs	Records sent reminders to avoid duplicates

4. Development Process

We followed Scrum methodology throughout the project, working in four weekly sprints. The sprint structure from our proposal held up well in practice.

- **Sprint 1:** focused on foundations: setting up the Docker environment (React, FastAPI, PostgreSQL), designing the database schema, building the Figma

wireframes, and implementing user authentication including Google OAuth 2.0 and JWT-based login.

- **Sprint 2:** delivered the Ethiopian–Gregorian date conversion engine and the core event CRUD system. The conversion uses the `ethiopian_date` Python library as its base, with our own wrapper that handles edge cases like the Pagume 13th month (5 days in a normal year, 6 in a leap year) and the New Year boundary (September 11 in a Gregorian leap year, September 12 otherwise).

- **Sprint 3:** was the most algorithmically demanding sprint. We implemented shared event collaboration with optimistic locking, the version conflict detection system, field-level diffs, participant conflict detection, the interval merging and gap-finding scheduling engine, and the constraint-based slot ranking algorithm. We also added email reminders via a background daemon thread.

- **Sprint 4:** focused on UI polishing, finalising the test suite, and writing this report. We added the Postman integration test collection (93 tests), extended the Pytest unit test suite to 116 tests, generated UML diagrams, and refined every page of the frontend.

5. Feature Implementation

5.1. Data and Database-Oriented Features

5.1.1. User accounts and profiles

Users register with an email and password, which is stored as a bcrypt hash, never in plaintext. The profile stores their preferred calendar (Ethiopian or Gregorian), timezone, and Google OAuth connection status. All profile fields can be updated via a 'PUT /profile' endpoint.

5.1.2. Event management

The full event CRUD is implemented across ‘POST’, ‘GET’, ‘PUT’, and ‘DELETE /events’ endpoints. Events support simple recurrence (daily or weekly), filtering by date range, full-text search across title and description, and an ownership filter (all events, owned only, or shared only). Event times are always stored in UTC in the database and converted to the user's local timezone on the way out.

5.1.3. Holiday data management

Holidays for both calendar systems are stored in the ‘holidays’ table. A seed function runs at startup to populate default public holidays. Users can also create and delete their own holidays via the API, and the system automatically converts Ethiopian dates to their Gregorian equivalents for consistent storage in the ‘resolved_date’ field.

5.2. Third-Party Service Integrations

5.2.1. Google OAuth 2.0

We implemented two separate OAuth flows: one for login (using ‘GET /auth/google/login-url’ and ‘GET /auth/google/callback’) and one for connecting an existing account to Google Calendar (using the ‘/google/’ router). The Google Calendar export feature allows users to push an existing Zemen event directly to their Google Calendar via ‘POST /google/export-event/{event_id}’.

5.2.2. Email reminders

A background daemon thread starts when the application boots and polls the database every 10 seconds. For each event whose ‘reminder_minutes’ window is approaching, the system sends an email to the event owner using Python's ‘smtplib’ with TLS. Once sent, the reminder is logged in ‘reminder_logs’ so it is never sent twice, even if the daemon restarts.

5.3. Complex Functionalities

5.3.1. Shared Events with Roles and Concurrent Conflict Detection

Events can be shared with other users via ‘POST /events/{event_id}/share’, assigning each participant a role: owner, editor, or viewer. Owners can share and manage participants; editors can modify event details; viewers can only read. This is enforced at every relevant endpoint.

The more interesting problem is what happens when two editors have the same event open at the same time and both try to save changes. This is the classic concurrent write problem, and we solved it with optimistic locking.

Every event has a ‘version’ integer field (starting at 1, incrementing with each update). When a client loads an event, it receives the current version number. When it saves, it must send that version back in the request body. The backend compares the submitted version against the current database version:

- If they match: the update proceeds, the version increments, and a new snapshot is written.
- f they do not match: the backend returns a ‘409 Conflict’ response containing ‘current_version’, ‘your_version’, and code: "VERSION_CONFLICT".

The frontend handles this gracefully, showing the user a message that the event was modified by someone else, with an option to view what changed (via the diff endpoint) or force-save their version anyway.

5.3.2. Field-Level Diff and Version History

Every successful event update writes an ‘EventSnapshot’, an immutable record of the event's state at that version. The ‘GET /events/{event_id}/diff?from_version=X&to_version=Y’ endpoint retrieves two snapshots, compares them field by field, and returns a structured diff:

“json

{

"changes": [

{ "field": "title", "from": "Team Sync", "to": "Team Sync Q2" },

{ "field": "reminder_minutes", "from": 30, "to": 60 }

],

"digest": "sha256-hash-of-change-summary"

}”

This gives users a clear, auditable record of every change made to a shared event, similar to the change history in Google Docs.

5.3.3. Scheduling Algorithm: Interval Merging, Gap Finding, and Constraint-Based Ranking

This is the most algorithmically complex part of the system. When an organiser wants to find a meeting time that works for all participants, the system runs a multi-stage pipeline:

Stage 1: Collect busy intervals

For each participant, query all their events within the search window. Each event becomes a ‘(start, end)’ tuple.

Stage 2: Merge intervals

The collected intervals often overlap or touch. The ‘merge_intervals’ function normalises and merges them into a minimal non-overlapping set. For example, three intervals ‘(09:00, 10:30)’, ‘(10:00, 11:00)’, ‘(11:00, 12:00)’ merge into a single ‘(09:00, 12:00)’ block.

Stage 3: Find gaps

Given the merged busy set and the search window boundaries, ‘find_gaps’ identifies the free intervals where a meeting could fit.

Stage 4: Generate candidates

‘choose_slots’ walks through each gap and generates all possible slots of the requested duration, advancing in 30-minute increments.

Stage 5: Rank by constraints

This is handled by the ‘rank_slots’ function, which scores each candidate slot using a penalty system:

Condition	Penalty
Required participant is busy	Slot excluded entirely
Optional participant is busy	+100 points
Slot is outside work hours	+30 points
Earliness preference	+N points (minutes from window start ÷ 10)

The ranked results are returned sorted by score ascending; lowest score means best slot. The frontend displays them visually, and the user can click any slot to populate the event form automatically.

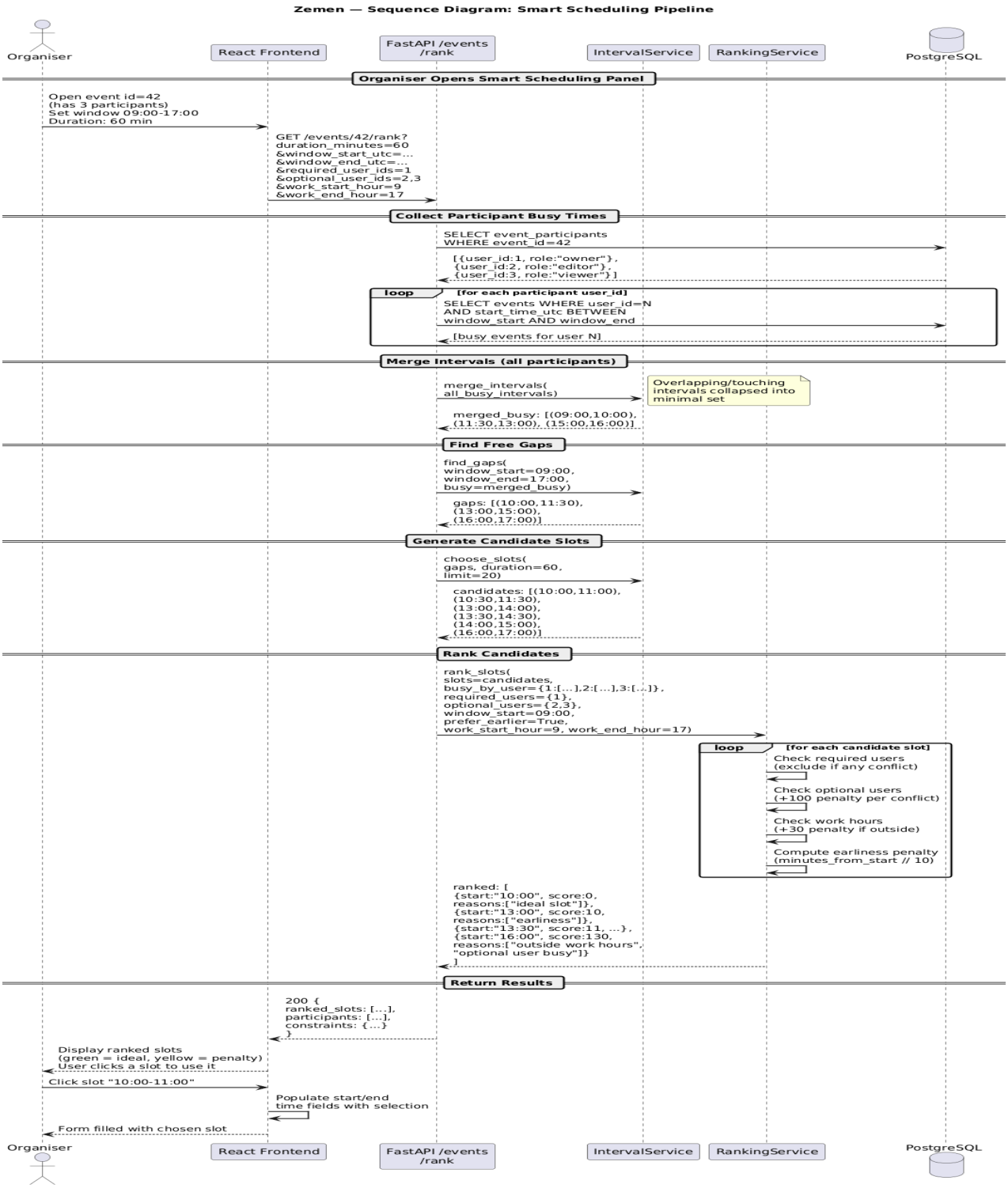

Figure 2: Sequence diagram showing the full smart scheduling pipeline from participant busy-time collection through interval merging, gap finding, slot generation, and constraint-based ranking.

This pipeline handles real edge cases: participants with no events get zero busy intervals; a fully blocked window returns an empty result; gaps shorter than the requested duration are skipped; and holidays are treated as all-day busy blocks for everyone.

6. UML Diagrams

We produced six UML diagrams for this project using PlantUML:

- **Class Diagram** - all six database models with fields, types, and relationships, plus the six core service classes with their method signatures.

- **Sequence Diagram: Authentication** - the full register → login → JWT → authenticated request flow, including the Google OAuth path and the alt blocks for invalid credentials and expired tokens.

- **Sequence Diagram: Create and Share Event** - event creation, snapshot writing, participant sharing, and how a shared event appears on a participant's calendar.

- **Sequence Diagram: Optimistic Locking and Version Conflict** - two editors loading the same event, one saving successfully, the other receiving a 409 conflict, reviewing the diff, and choosing to force-save.

- **Sequence Diagram: Smart Scheduling Pipeline** - shown in Figure 2 above.

- **Component Diagram** - the full system architecture shown in Figure 1 above.

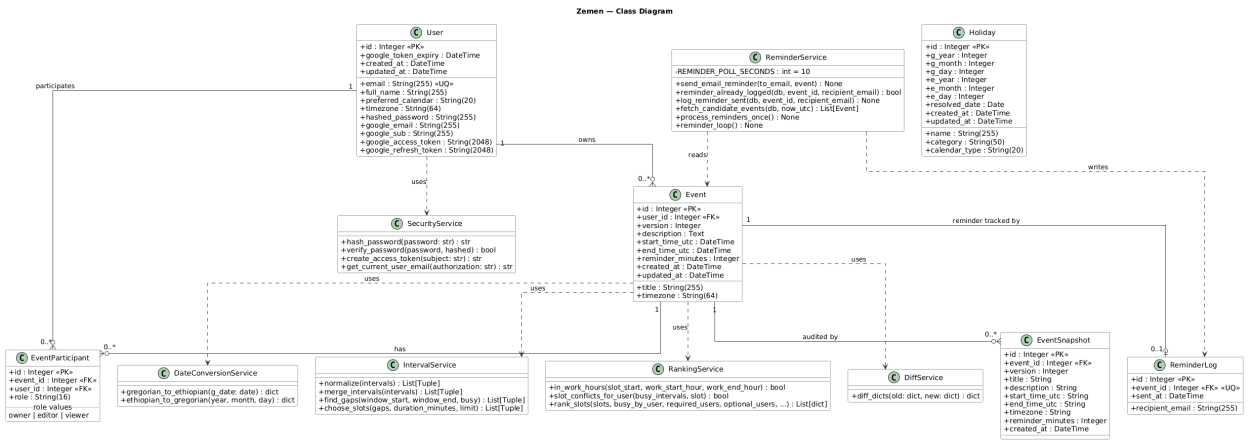

Figure 3: Class diagram showing the six database models and six core service classes with their relationships.

7. Testing

We took testing seriously from Sprint 2 onwards. Every major algorithm and every API endpoint has test coverage.

7.1. Unit Tests - Pytest

We wrote 116 unit and integration tests across two test files, run with Pytest on Python 3.12.

‘test_date_conversion_edge_cases.py’ (16 tests) covers the Ethiopian–Gregorian conversion engine in detail: the New Year boundary on September 12 (non-leap year) and September 11 (Gregorian leap year), Pagume having 5 days in a normal Ethiopian year and 6 in a leap year, six parametrised round-trip conversions, year-boundary dates, and a test that verifies all 13 Ethiopian month names are reachable through the conversion function.

‘test_scheduling_pipeline.py’ (31 tests) covers the full scheduling algorithm end-to-end: two-user conflict scenarios, a fully-blocked window returning no results, fragmented

intervals that merge into a single usable gap, required vs. optional participant handling, sort-order guarantees, version conflict detection logic, role permission checks, and edge cases like a gap shorter than the requested duration.

Figure 4: Full Pytest run output showing 116 tests collected and passed with zero failures.

7.2. Integration Tests - Postman

We built a Postman collection with 93 tests across 10 folders, covering every endpoint in the API. The collection is designed to run top-to-bottom as a full integration workflow:

- **Health:** Verifies the server is up
- **Authentication:** Register (using a timestamp-generated unique email to support re-runs), login, wrong password (expect 401), and no-token access (expect 401/422)
- **Profile:** Get and update profile
- **Date Conversion:** Gregorian to Ethiopian, Ethiopian to Gregorian, and missing parameter validation (expect 422)
- **Events CRUD:** Create, list, get by ID, update (with live version fetch pre-request), version conflict (expect 409), non-existent event (expect 404)
- **Sharing and Participants:** Share event, list participants
- **Scheduling:** Conflict detection, time slot suggestion, ranked slot results
- **Event Diff:** Field-level change history between versions

- **Holidays:** List, create, delete
- **Google Calendar:** Connection status, connect URL
- **Cleanup:** Delete test event

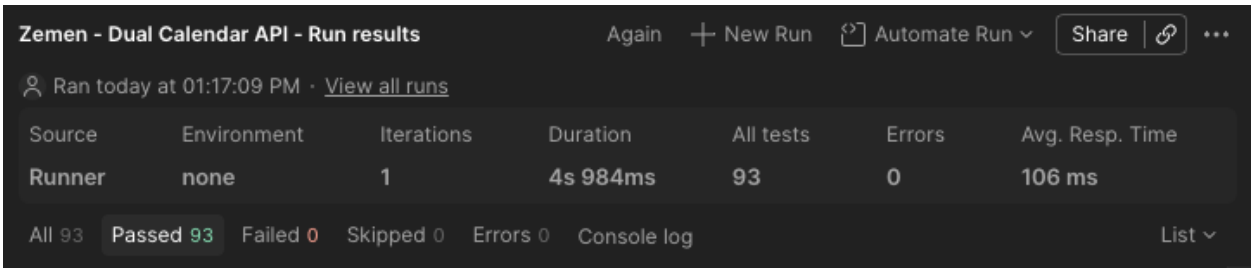

Figure 5: Postman Collection Runner results showing all 93 tests passing across 10 folders.

A key design decision in the Postman collection was making it fully rerunnable: the registration step uses a 'Date.now()' timestamp in the email address so duplicate registration errors never occur on subsequent runs. The update event step uses a pre-request script that fetches the current event version before sending the update, so the version number is always fresh.

8. Challenges and How We Solved Them

8.1. Ethiopian calendar edge cases

The conversion engine required careful testing around the New Year boundary and the Pagume month. September 11 is Ethiopian New Year in a Gregorian leap year; September 12 in all other years. Pagume has 5 days normally and 6 in an Ethiopian leap year (which follows the rule: year % 4 == 3). We caught several off-by-one errors through the round-trip parametrised tests.

8.2. Optimistic locking and stale versions in Postman

When we first wrote the Postman collection, the Update Event request was sending a hardcoded version number that quickly became stale after the first run. We solved this by adding a pre-request script that calls ‘pm.sendRequest’ to fetch the current event version live, storing it as a collection variable, and using that value in the update body.

8.3. FastAPI header validation

The login endpoint takes ‘email’ and ‘password’ as query parameters (bare function arguments in FastAPI), not a JSON body. This caused a cascade failure in the Postman collection where the login request was sending a JSON body, getting a 422 back, the token was never saved, and every subsequent authenticated request failed. Once we corrected the login request to use query parameters, the entire collection passed.

8.4. Concurrent scheduling with holidays

The scheduling engine treats holidays as an all-day busy interval for all participants. This required merging holiday intervals into the busy set before computing gaps, which we handled inside ‘busy_intervals_for_user’ by calling ‘holiday_intervals’ and extending the intervals list before passing it to ‘merge_intervals’.

8.5. Security considerations

For a university project, the security baseline is solid; passwords are hashed with bcrypt, JWTs expire after 60 minutes, role-based access is enforced at every endpoint, and SQLAlchemy ORM prevents SQL injection. For a production deployment, three things would need addressing: the login endpoint currently passes credentials as query parameters which appear in server logs and should move to the request body; the default SECRET_KEY fallback should be removed so the app refuses to start without an explicit environment variable; and Google OAuth tokens stored in the database should be encrypted at rest rather than stored as plain strings.

9. Conclusion

Zemen turned out to be a much more complete and technically interesting system than we originally proposed. Starting from what the professor rightly pointed out was a mostly-CRUD application, we extended it into something with real algorithmic substance: a concurrent editing model with optimistic locking, a full scheduling pipeline based on interval merging and constraint-based ranking, and a field-level change-tracking system. These are not toy implementations; they are the same patterns used in the production of collaborative tools.

Everything we committed to in the approved proposal was delivered:

Feature	Status
User accounts and profile CRUD	Done
Event management with recurrence	Done
Holiday data (both calendars)	Done
Google OAuth 2.0	Done
Email reminders (SMTP)	Done
Ethiopian–Gregorian conversion engine	Done
Timezone-aware scheduling	Done
Shared events with roles (owner/editor/viewer)	Done
Optimistic locking and version conflict detection	Done
Field-level diff and version history	Done
Interval merging and time slot suggestion	Done
Constraint-based slot ranking algorithm	Done
Figma wireframes	Done
UML diagrams	Done (6 diagrams)
Pytest unit tests	Done (116 tests, all passing)
Postman integration tests	Done (93 tests, all passing)
Docker Compose deployment	Done

The codebase is clean, modular, and well-tested. We are confident it demonstrates the kind of software engineering depth the project required.

10. Future Plans

Mobile app : A React Native version of the frontend would make the app much more accessible for everyday use.

Natural language event input: Parsing phrases like "Meet John next Tuesday at 2pm Addis Ababa for 1 hour" into structured event data was in an early draft of our proposal. It remains a strong extension for a future version.

Real-time collaboration: The current optimistic locking model works well but requires a page reload to see another user's changes. WebSocket-based real-time updates would make the collaboration experience much smoother.

Calendar sync: Full bidirectional Google Calendar sync, not just export, would complete the Google integration story.